캡스톤 4분반 · 7조

게임 리뷰 AI 요약 서비스

프로젝트 결과보고서

팀 7조

팀원 임상진 · 임석주 · 노현준 · 김경열

작성일 2026년 11월

캡스톤 페스티벌 2026 제출 자료

요약

게임 리뷰 AI 요약 서비스는 Steam과 Metacritic에 흩어진 게임 리뷰를 근거 단위로 구조화하고, 게임의 장단점, 유저와 평론가의 평가 차이, 플레이타임별 여론을 한 화면에 제공하는 웹 서비스다. 평균 평점만으로는 알기 어려운 구매 판단 정보를 정리하는 것이 목적이며, 캡스톤 4분반 7조의 임상진, 임석주, 노현준, 김경열이 개발했다.

핵심 설계는 리뷰에서 근거를 뽑는 Map 단계와 근거를 종합하는 Reduce 단계의 분리다. Map은 리뷰마다 리뷰 ID·평가 항목·긍부정·구체적 내용을 JSON 근거로 만들고, Reduce는 이 근거로 유저·평론가·플레이타임·통합 요약을 작성한다. 점수는 Steam 추천률 같은 공식 통계를 기준값으로 삼고, LLM은 인용 검증을 통과한 보정값만 제안한다. Map은 로컬 GPU(Ollama) 또는 Groq API에서, Reduce는 Groq API에서 실행된다.

2026년 6월 발표 시점에 저장된 평가 결과는 다음과 같다. Gemini가 판정한 통합 요약의 근거 충실도는 100게임 평균 0.931이었다. Map 결과를 고정하고 Reduce를 5게임에서 5회씩 반복했을 때 총점(0~100) 표준편차 평균은 0.98이었다. 코어 평가셋 20게임의 점수와 Steam 추천 분포의 정합도는 총점 0.909, 항목 0.842였다. 단일 GPT-5.5 판정자가 단순 단일 프롬프트 요약과 비교한 유용성 평가는 베이스라인 29 대 파이프라인 11로 베이스라인이 우세했다. 사용자 스토리 15개 범주 중 완전 달성 5개, 부분 달성 7개, 미달 3개다.

보고서 작성 시점(2026-11-10)에는 위 수치를 저장된 원자료에서 다시 계산해 일치함을 확인했고, LLM을 호출하지 않는 회귀 테스트 102개가 로컬에서 모두 통과했다. LLM 요약 생성, 크롤링, 배포는 다시 실행하지 않았다. 근거를 출처까지 추적하는 요약 구조를 끝까지 구현하고 품질을 네 가지 관점으로 나누어 측정한 것이 주요 성과다. 반면 구매 판단에 필요한 정보의 폭, 카탈로그 자동 확장, 비용 측정은 다음 과제로 남았다.

목차

1. 배경과 문제 정의 **4**
- 1.1 리뷰 탐색의 어려움 4
- 1.2 보고서의 범위와 시점 4

2. 목표·요구사항·구현 범위 **4**
- 2.1 요구사항 4
- 2.2 구현 범위의 변화 5
- 2.3 품질 목표와 검증 방법 5

3. 전체 시스템 구조와 기술 선택 **6**
- 3.1 구성 요소와 책임 6
- 3.2 기술 선택 7
- 3.3 두 가지 실행 경로 7

4. 데이터 수집과 AI 처리 파이프라인 **8**
- 4.1 수집·정제·적재 8
- 4.2 표본 선택과 플레이타임 구간 8
- 4.3 Map: 근거 추출과 복구 8
- 4.4 Reduce와 결과 저장 9

5. 점수 산출과 운영 신뢰성 **10**
- 5.1 공식 통계를 기준으로 한 점수 10
- 5.2 레이더 점수와 대표 강·약점의 분리 10
- 5.3 증분 갱신의 품질 보호 11
- 5.4 구매 신호·번역·추천 챗봇 11

6. 주요 기능과 실제 시연 화면 **12**
- 6.1 게임 목록과 상세 화면 진입 12
- 6.2 유저·평론가 요약과 대표 리뷰 13
- 6.3 장단점·항목별 분석·플레이타임별 여론 14
- 6.4 구매 타이밍과 추천 대상 16

7. 개발 과정과 기술적 문제 해결 **17**
- 7.1 협업과 스프린트 17
- 7.2 주요 기술 문제와 해결 17
- 7.3 발표 이후 변경 18

8. 실험·검증 방법과 결과 **18**
- 8.1 평가 대상과 조건 18
- 8.2 근거 충실도 18
- 8.3 점수 안정성 19
- 8.4 점수와 추천 분포의 정합성 20
- 8.5 유용성 비교 20
- 8.6 회귀 테스트 20

9. 성과와 목표 달성도 **21**
- 9.1 요구사항별 달성 결과 21
- 9.2 기술적 성과 21

10. 한계와 향후 개선 **22**
- 10.1 데이터와 출력 품질 22
- 10.2 평가의 한계 22
- 10.3 기능과 운영 22

11. 결론 **22**

12. 참고자료 **23**

1. 배경과 문제 정의

1.1 리뷰 탐색의 어려움

게임을 사기 전 사용자는 리뷰를 찾아보지만, 인기작 하나에도 리뷰가 수백~수천 개 달리고 길이와 내용이 제각각이다. 평균 평점 하나로는 구체적인 장점과 단점, 오래 플레이해도 재미가 유지되는지, 평론가와 일반 유저의 평가가 갈리는지, 지금이 살 만한 때인지를 알 수 없다(README §1). 같은 리뷰 안에서도 그래픽을 칭찬하면서 최적화를 비판하는 식으로 항목별 평가가 엇갈리기 때문에, 추천·비추천 하나로 정리하면 이런 차이가 사라진다.

README는 평론가와 일반 유저의 평가가 갈리는지, 오래 플레이해도 재미가 유지되는지를 평균 평점으로는 알 수 없는 정보로 꼽는다(README §1). 이를 하나의 요약으로 합치면 관점과 시점의 차이가 흐려진다.

이 프로젝트는 리뷰를 짧게 줄이는 것보다 **어떤 평가가 어느 리뷰에서 나왔는지 추적할 수 있게 만드는 것**을 핵심 문제로 삼았다. 단순히 리뷰 묶음을 LLM에 넣어 요약하는 방식은 근거를 확인할 수 없고, 다른 게임 이야기를 지어내거나 실행마다 점수가 흔들리는 문제가 있었다(발표자료 슬라이드 7~8). 그래서 원문과 최종 문장 사이에 리뷰 ID를 가진 근거 계층을 두고, 이 근거를 요약·점수 보정·출처 분리·사후 품질 평가에 함께 사용했다. README가 정리한 차별점(근거 기반 구조화, 유저·평론가 분리, 플레이타임 구간 분석, 항목별 평가, 점수 출처 분리, 구매 시점 신호)은 모두 이 근거 계층 위에서 동작한다(README §2).

1.2 보고서의 범위와 시점

이 보고서는 2026년 6월 최종 발표 당시의 구현·실험 결과와, 보고서를 작성한 2026-11-10의 재확인 결과를 구분해 쓴다. 재확인은 저장된 실험 결과 파일의 재계산, 코드·설정 확인, 로컬 회귀 테스트 재실행으로 한정했다. LLM 요약 생성·크롤링·서비스 배포는 다시 실행하지 않았으므로, 실험 수치는 모두 발표 당시 생성된 결과다.

발표 당시 서비스는 Cloudflare Tunnel 임시 주소로 공개되었으나, 2026-11-10에는 그 주소가 응답하지 않았다. 따라서 6장의 화면은 2026-06-07 저장소에 커밋된 실제 서비스 캡처이며, 현재 공개 서비스의 동작을 보여 주는 증거는 아니다.

2. 목표·요구사항·구현 범위

2.1 요구사항

프로젝트의 목표는 여러 플랫폼의 리뷰를 수집·정제하고, 근거를 추적할 수 있는 요약과 분석을 사용자에게 제공하는 것이다. 학기 초 Product Backlog는 수집·DB 저장·AI 요약·목록/상세 화면을 핵심 경로로 두고, 검색·필터·번역과 부가 기능으로 확장하는 구성이었다. 백로그에서 중복된 검색·자동완성 항목을 하나로 합치면 표 1의 15개 범주가 된다. 구현 결과는 9장에서 같은 이름으로 대조한다.

표 1. 학기 초 요구사항 범주와 우선순위

요구사항	우선순위	요구한 동작(인수 조건 요약)
리뷰 데이터 수집 파이프라인	High	Steam·Metacritic 리뷰 수집, 실패 시 재시도
수집 데이터 정제 및 DB 저장	High	텍스트 정제, 중복 없는 저장
AI 기반 리뷰 핵심 요약 생성	High	긍정·부정 핵심 정보와 자연스러운 요약
게임 목록 및 상세 페이지	Medium	카드 목록과 게임별 요약 화면 연결
다국어 리뷰 자동 번역	Medium	외국어 리뷰의 한국어 번역 표시
게임명 통합 검색 및 자동 완성	Medium	자동완성 드롭다운과 상세 이동
상세 조건 필터링 탐색	Medium	장르·점수 조건 필터, 빈 결과 안내
리뷰 출처 및 신뢰도 배지	Medium	개별 리뷰의 출처·플레이타임 표시
텍스트 위주(무평점) 리뷰 자동 요약	Medium	무평점 리뷰의 1~2줄 요약
사용자 취향 기반 개인화 추천	Low	취향 저장과 개인화 목록
인기/신작 게임 주기적 자동 동기화	High	인기작·신작 탐색과 신규 등록
리뷰 필터 및 정렬	Medium	리뷰 단위 별점·최신순·출처 필터
플랫폼 내 자체 유저 리뷰 작성	Low	로그인 후 별점·리뷰 등록
실시간 게임 인기 차트	Medium	자체 순위와 순위 변동 표시
검색 기반 게임 정보 실시간 등록	High	DB에 없는 게임의 즉시 등록

2.2 구현 범위의 변화

일부 기능은 개발 과정에서 형태가 바뀌었다. 플레이타임은 개별 리뷰 배지 대신 구간별 여론 분석으로 확장되었고, 취향 추천은 취향을 저장하는 마이페이지 대신 선호·비선호 게임을 대화로 입력받는 추천 챗봇으로 구현되었다. 목적은 일부 충족하지만 원래 인수 조건과 다르므로 부분 달성으로 분류한다. 게임 비교 화면, 플레이타임별 여론, 구매 타이밍 신호, 외부 품질 평가는 계획에 없던 추가 결과물이다. 반면 회원·로그인과 자체 리뷰, 실시간 인기 차트, DB에 없는 게임의 즉시 등록은 구현하지 못했다 (PRESENTATION §5).

2.3 품질 목표와 검증 방법

"AI 요약"의 품질은 하나의 수치로 판단하기 어렵다. 팀은 품질 목표를 다섯 가지로 나누고 각각 다른 방법으로 검증했다(표 2). 회귀 테스트는 LLM 출력을 받아 처리하는 코드가 올바른지를, LLM 판정 실험은 생성된 결과물이 근거에 맞고 쓸모 있는지를 본다. 어느 한쪽의 통과만으로 전체 품질을 말할 수 없기 때문에 둘을 함께 사용했다.

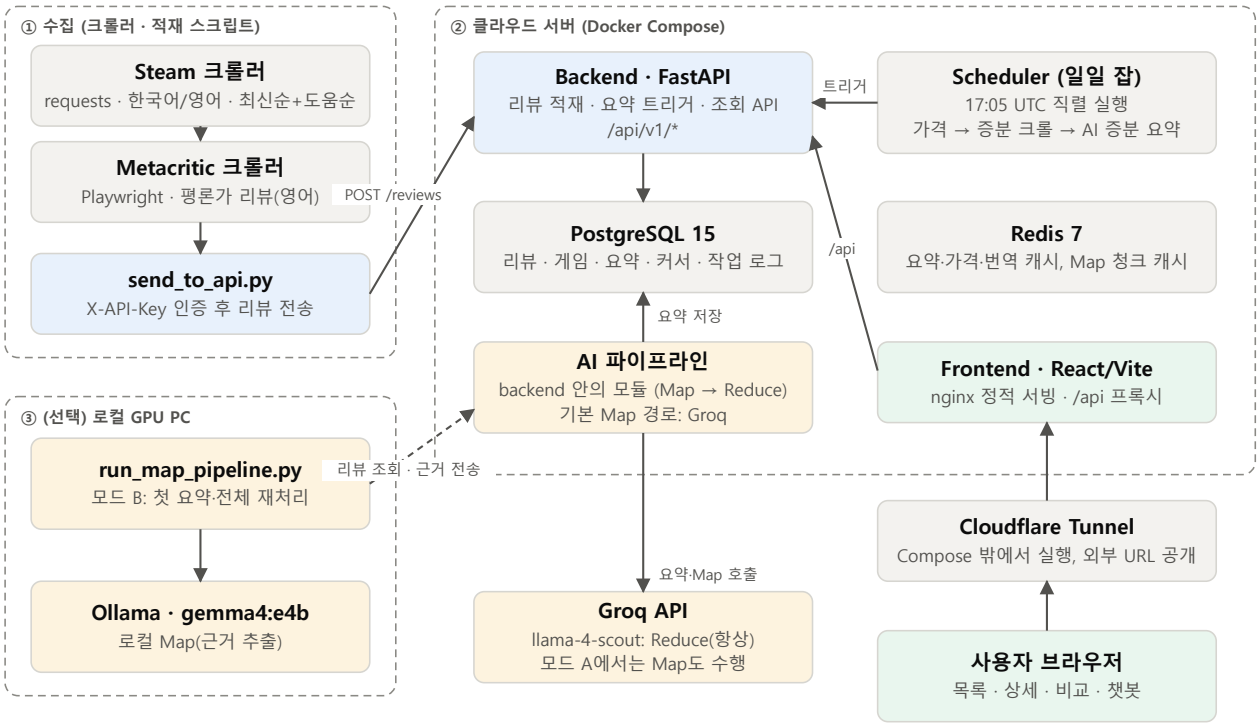
표 2. 품질 목표와 검증 방법

품질 목표	검증 방법	결과 위치
요약 문장이 실제 리뷰 근거에 맞는다	RAGAS 근거 충실도(독립 판정 모델) + 근거 바꿔치기 대조 실험	8.2
같은 근거에서 점수가 크게 흔들리지 않는다	저장된 Map 결과로 Reduce 반복 실행	8.3
점수가 실제 추천 분포와 맞는다	점수와 Steam 추천율의 산술 비교	8.4
사용자에게 더 유용하다	단순 요약과 블라인드 쌍 비교(LLM 판정)	8.5
깨진 LLM 출력을 안전하게 처리한다	LLM 없이 합성 입력을 넣는 회귀 테스트	8.6

3. 전체 시스템 구조와 기술 선택

3.1 구성 요소와 책임

시스템은 수집기, 백엔드, AI 파이프라인, 저장소, 프론트엔드, 스케줄러로 구성된다(그림 1). 수집기는 외부 리뷰를 가져와 백엔드 적재 API로 보내고, PostgreSQL은 원문 리뷰와 구조화된 요약을 저장한다. Redis는 다시 만들 수 있는 중간 결과(Map 청크, 이전 근거)와 조회 캐시, 가격·여론 스냅샷을 보관한다. 프론트엔드는 생성된 결과를 조회해 화면을 구성한다. 스케줄러는 API와 분리된 컨테이너로, 매일 17:05 UTC에 가격·여론 갱신, 증분 크롤링, AI 증분 요약을 순서대로 실행한다(ARCHITECTURE §1·§12).



실선: 기본 경로(모드 A) · 점선: 선택 경로(모드 B, Tunnel URL을 통해 백엔드 호출) · 주황: LLM 처리

README §5·§7, ARCHITECTURE §1·§2·§12, docker-compose.yml을 근거로 재구성한 설명 도식

그림 1. 수집·저장·AI 처리·화면 제공의 전체 구조 (재구성 도식)

3.2 기술 선택

학기 초 기술 결정 기록(ADR)에는 FastAPI, PostgreSQL+JSONB, React+Tailwind, API와 Playwright를 병행한 수집, 상용 LLM API+캐싱이 채택되어 있다. 최종 시스템은 이 선택 위에 Map·Reduce 분리와 로컬·클라우드 실행 경로를 더했다(표 3).

표 3. 주요 기술과 역할

영역	기술	역할
화면	React, Vite, React Router, Tailwind, nginx	목록·상세·비교 화면, API 프록시
API	FastAPI, SQLAlchemy(asyncpg)	비동기 조회, 리뷰 적재, 요약 실행 조정
저장	PostgreSQL 15(JSONB)	리뷰 관계 데이터와 구조화 요약
캐시	Redis 7	청크·요약·가격·여론·챗봇 카탈로그
수집	requests, Playwright	Steam API 수집, Metacritic 웹 수집
AI	Ollama(gemma4:e4b), Groq API(llama-4-scout)	로컬 근거 추출, 클라우드 요약
운영	Docker Compose, Cloudflare Tunnel	서비스 구성과 외부 공개

게임·플랫폼·리뷰처럼 식별 관계가 분명한 데이터는 관계 테이블에 두고, 장단점·키워드·항목별 근거처럼 구조가 중첩되는 요약 결과는 JSON 필드에 저장했다. 원문과 요약을 분리하면 요약을 다시 생성해도 출처 리뷰가 유지된다(ARCHITECTURE §7).

3.3 두 가지 실행 경로

Map은 리뷰 원문 전체를 다루므로 입력 토큰이 가장 많고, Reduce는 압축된 근거를 받아 한국어로 종합하므로 품질이 중요하다. 팀은 이 차이에 맞춰 두 실행 경로를 만들었다(표 4). 대량 처리는 Map을 로컬 GPU에서 돌려 Groq API의 토큰 한도와 사용량 부담을 피하고, 일상 증분은 GPU 없이 클라우드만으로 처리하려는 설계다(ARCHITECTURE §1~2). 단, 이 경로 분리로 실제 토큰·비용이 얼마나 줄었는지는 측정하지 않았다.

표 4. 실행 경로별 Map·Reduce 위치 (현재 설정 기준)

경로	시작 방식	Map	Reduce	용도
모드 A	백엔드 요약 API(스케줄러 포함)	Groq(기본값 MAP_BACKEND=groq)	Groq	일상 증분 요약
모드 B	로컬 실행기 run_map_pipeline.py	로컬 Ollama 또는 Groq(자동 분배 가능)	백엔드의 Groq	첫 요약, 전체 재처리

Groq 모델의 기본값은 llama-4-scout-17b-16e-instruct이며, Groq Map용 모델을 따로 지정하지 않으면 Reduce와 같은 모델을 쓴다(ARCHITECTURE §13). 로컬 Map 기본 모델은 gemma4:e4b다. 모드 B에서 첫 실행이나 전체 재처리를 하면 Reduce 입력 기록(payload)이 보존된다. 이 기록으로 Map 없이 Reduce만 다시 실행할 수 있어, 요약·점수 로직을 바꿀 때 근거 추출과 최종 조합을 분리해 점검할 수 있다. 8.3의 안정성 실험도 이 기능을 사용했다(MAP_REDUCE §8).

4. 데이터 수집과 AI 처리 파이프라인

4.1 수집·정제·적재

Steam 수집기는 한국어·영어 리뷰를 최신순과 도움순으로 함께 가져온다. 최신순은 최근 패치나 할인 유입의 반응을, 도움순은 오래 공감을 받은 핵심 평가를 확보하기 위한 선택이며, 두 목록의 중복은 제거한다. 리뷰마다 작성 시점 플레이타임, 도움 수, 문장 단위 카테고리 태그를 함께 저장한다. 수집 단계에서 이모지와 반복 특수문자를 정리하고 10자 미만 리뷰를 제거한다(steam_crawling 문서). Metacritic 평론가 리뷰는 Playwright로 수집한다.

적재 API는 점수를 0~100 범위로 정규화한다. Steam은 추천 여부, Metacritic 평론가는 100점, 유저는 10점(×10)을 사용한다. 리뷰는 (플랫폼, 게임, 원본 리뷰 키)를 유일 키로 저장하므로, 같은 리뷰가 다시 들어오면 갱신만 되고 새 리뷰만 새 ID를 받는다. 요약 단계는 마지막으로 처리한 리뷰 ID(커서) 이후의 리뷰만 처리하므로, 매일 같은 범위를 다시 수집해도 중복 처리가 생기지 않는다(ARCHITECTURE §3).

4.2 표본 선택과 플레이타임 구간

인기작의 리뷰를 모두 LLM에 넣을 수는 없으므로 게임마다 표본을 고른다. 요약 입력의 목표값은 기본 200건이다. 샘플러는 플랫폼 비율과 추천·비추천 비율에 따라 표본 수를 나누고, 각 묶음 안에서는 도움 수와 플레이타임으로 계산한 품질 점수가 높은 리뷰를 고른다(MAP_REDUCE §2). 언어는 한국어·영어를 우선하지만 필터 후 50건 미만이면 전체 언어로 되돌린다. 요약의 균형을 위해 반대 의견 근거를 보강하는 절차도 있어서, 표본의 긍부정 비율은 원본과 다를 수 있다.

플레이타임 구간은 게임마다 리뷰어 플레이타임 분포의 하위 33%·66% 지점(p33, p66)으로 나눈다. 게임마다 플레이 시간 규모가 다르기 때문에 고정 구간 대신 분포 기반 구간을 택했다. 구간을 만들려면 플레이타임이 있는 Steam 리뷰가 전체 18건 이상 필요하다. 품질 점수 계산에서는 플레이타임을 500시간으로 잘라 극단값의 영향을 줄이고, 구간별 가중치는 초반 0.4, 중반 0.7, 후반 0.5, 정보 없음 0.3을 준다(sampler.py; playtime_improvement 문서). 중반 구간을 가장 균형 잡힌 경험으로 본 설계 판단이며, 이 가중치가 더 나은 요약을 만든다는 비교 실험은 없다.

4.3 Map: 근거 추출과 복구

고른 리뷰는 약 1,400자 단위의 청크로 묶고, 문맥이 끊기지 않게 앞 청크의 리뷰를 최대 2개 겹쳐 넣는다. 청크에는 리뷰 ID, 도움 수, 플레이타임을 본문과 함께 넣어 출처 연결의 출발점으로 삼는다(MAP_REDUCE §1). 그림 2는 이후 근거 추출부터 저장까지의 흐름이다.

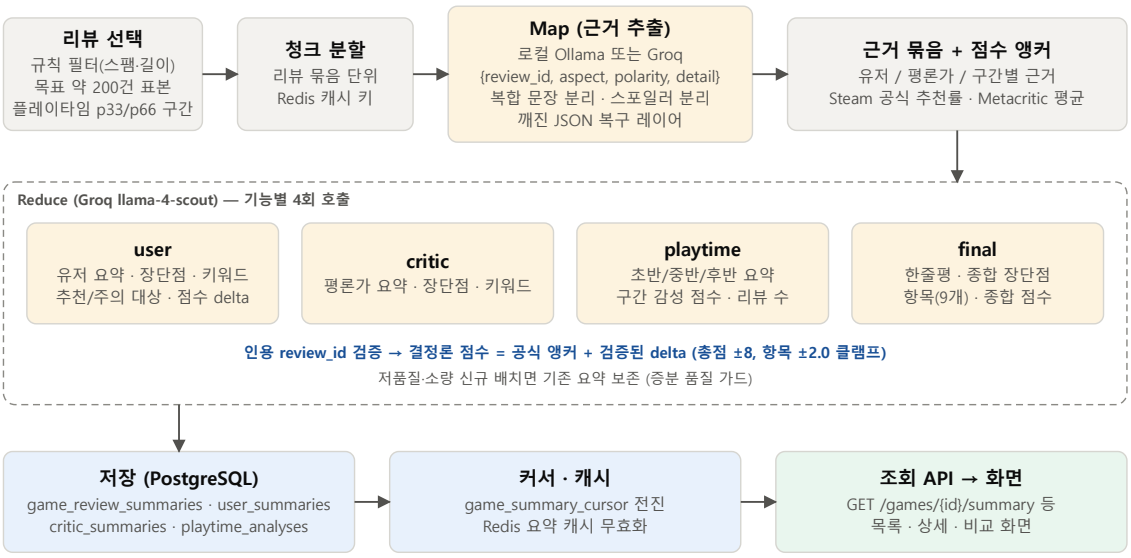


그림 2. 표본 선택에서 근거 추출·요약·저장까지의 흐름 (재구성 도식)

Map은 각 근거를 그래픽, 조작감, 최적화, 콘텐츠, 스토리, 가성비, 사운드, 게임플레이, 난이도의 9개 항목 중 하나로 분류한다. "그래픽은 좋은데 프레임이 떨어진다"처럼 한 문장에 두 평가가 섞이면 근거 두 개로 나눈다. 긍부정은 특성의 강도가 아니라 만족도로 판단하게 해서, "어렵지만 재밌다"를 단점으로 분류하지 않도록 했다(map_local.py). 근거에는 리뷰 ID, 항목, 긍부정, 정제된 내용, 공개용 문장, 스포일러 위험 표시, 원문 인용이 함께 담긴다. 공개용 문장을 따로 두어 스포일러 고유명사를 가린다(MAP_REDUCE §3).

로컬 LLM은 형식이 다르거나 중간에 끊긴 JSON을 자주 출력했다. 복구 계층은 코드 블록 표시나 앞뒤 잡음을 지우고, 배열만 오거나 필드가 빠진 출력도 리뷰 ID를 기준으로 원래 형식으로 재구성한다. 이어서 청크에 실제로 있는 리뷰 ID와 허용된 항목인지 검사한다(MAP_REDUCE §4). 이 검사는 존재하지 않는 ID를 걸러내는 형식 검증이며, 그 리뷰가 실제로 그 내용을 말했는지는 8.2의 충실도 평가로 따로 확인했다.

4.4 Reduce와 결과 저장

Map 근거는 전체·유저·평론가·플레이타임 그룹으로 묶여 Reduce로 전달된다. Reduce는 기능별로 user(유저 요약·추천/주의 대상·점수 보정), critic(평론가 요약), playtime(구간별 요약), final(한줄평·종합 장단점·항목 점수)의 네 번 호출로 나뉜다. 한 번에 모든 결과를 요구하면 뒤쪽 구간이 빈 문장으로 채워지거나 유저 표현이 평론가 요약에 섞이는 문제가 있어서 분리했다(MAP_REDUCE §5). 결과는 표 5처럼 나누어 저장한다. 관점별 결과를 따로 두면 화면에서 서로 다른 근거와 점수를 나란히 보여 줄 수 있고, 갱신과 캐시 무효화 단위도 분명해진다.

표 5. 주요 데이터의 저장 위치

데이터	저장 위치	내용
원본 리뷰	external_reviews	원본 키, 추천 여부, 플레이타임, 태그
통합 결과	game_review_summaries	한줄평, 장단점, 항목 점수, 추천 대상
유저·평론가 결과	user_summaries, critic_summaries	관점별 요약과 독립 점수
플레이타임 분석	playtime_analyses	구간 경계, 리뷰 수, 추천 비율, 요약
증분 상태	game_summary_cursor	마지막으로 처리한 리뷰 ID
중간 근거·캐시	Redis	Map 청크, 이전 근거, 조회 결과
실행 기록	review_summary_jobs	실행 이력과 진단 정보

5. 점수 산출과 운영 신뢰성

5.1 공식 통계를 기준으로 한 점수

LLM이 점수를 자유롭게 내면 실행마다 흔들리고 근거와 연결되지 않는다. 그래서 점수의 기준값은 공식 통계로 고정하고, LLM에게는 인용 근거가 확인된 보정값만 제안하게 했다. 최종 점수는 코드가 계산한다(README §2).

항목 점수(0~10)는 전체 추천률에서 만든 기준값에 그 항목의 긍정·부정 근거 비율을 반영해 계산한다. 근거가 적은 항목은 기준값 쪽으로 당겨 소수 의견에 크게 흔들리지 않게 하고, 항목마다 다른 언급 경향은 사전 보정값으로 맞춘다. 근거가 전혀 없는 항목은 점수를 만들지 않는다. 종합·유저 점수(0~100)는 Steam 공식 추천률을 기준값으로 쓰고, 공식 통계 호출이 실패하면 수집한 표본의 추천률로 대신한다. 평론가 점수에는 LLM 보정 없이 Metacritic 평론가 평균을 사용한다(ARCHITECTURE §5). LLM 보정은 표 6의 조건을 모두 통과할 때만 반영된다. 발표자료의 "±2.0 보정"은 항목 점수의 범위다.

표 6. LLM 점수 보정의 적용 조건 (reduce_api.py 확인)

점수	보정 범위	적용 조건	최종 범위
항목 점수	−2.0 ~ +2.0	기준 근거 2건 이상, 유효한 리뷰 ID 인용 1개 이상	2.0 ~ 9.0
종합·유저 점수	−8 ~ +8	최소 표본(설계 문서 기준 10건) 이상, 유효한 인용 2개 이상	0 ~ 100
평론가 점수	보정 없음	Metacritic 평론가 평균 사용	100점 기준

경계는 코드가 정하지만 LLM이 제안하는 보정값은 실행마다 달라질 수 있으므로, 같은 입력에서 점수가 완전히 같다고 보장되지는 않는다(8.3).

5.2 레이더 점수와 대표 강·약점의 분리

상세 화면의 레이더는 콘텐츠·게임플레이·그래픽·조작감·최적화의 공통 5축으로 고정하고, 나머지 항목은 근거가 충분하고 두드러질 때만 "눈에 띄는 반응"으로 보여 준다. 레이더는 항목 점수의 크기를 보여 주지만, "대표 강점·약점" 문구는 전체 리뷰의 언급량과 긍정률로 따로 정한다. 강점은 언급 30건 이상·긍정률 0.92 이상, 약점은 언급 12건 이상·긍정률 0.78 이하다(ARCHITECTURE §5-1-1).

이 기준은 반복된 재설계의 결과다. 처음에는 표본 안의 긍부정 비율로 강·약점을 판정했는데, 조작감·최적화가 거짓 약점으로 표시되고 명작의 대표 강점은 빠지는 문제가 있었다. 팀은 대표 게임 9개의 실제 리뷰로 원인을 진단한 뒤 전체 리뷰의 언급량과 긍정률 기준으로 바꿨다(PRESENTATION §2, 챌린지 C). 다만 이 임계값은 특정 게임들로 조정한 값이며 전체 게임에서 최적이라고 검증되지는 않았다.

5.3 증분 갱신의 품질 보호

증분 요약은 커서 이후의 신규 리뷰만 Map에 보낸다. 신규 리뷰가 몇 건뿐인 상태로 요약을 다시 쓰면 기존의 좋은 요약이 소수 의견에 덮이고 점수가 흔들린다. 이를 막기 위해 세 가지 장치를 두었다. 신규 리뷰가 기본 8건 미만이면 요약과 커서를 그대로 두고 다음 실행까지 누적한다. 직전 근거를 Redis에 보관했다가 다음 Reduce에 합친다. 새 결과가 비었거나 실패하면 기존 요약을 덮어쓰지 않는다(ARCHITECTURE §5-7). Groq 무료 한도에 대해서는 429 응답의 대기 시간 존중, 여러 키의 순환 사용, 일일 한도 소진 시 즉시 실패 처리를 적용했다(PRESENTATION §2, 챌린지 B). 이 장치들이 실제 운영에서 얼마나 자주 동작했는지는 측정하지 않았다.

5.4 구매 신호·번역·추천 챗봇

구매 타이밍 신호는 할인율과 최근 월별 여론 변화(최소 20건이 있는 달 기준)를 함께 본다. 가격과 여론 스냅샷은 일일 잡이 Redis에 미리 채워 두고, 조회 API는 이 스냅샷만 읽어 사용자 요청이 외부 가격 API 호출로 이어지지 않게 했다. 가격 스냅샷이 28시간보다 오래되면 할인을 단정하지 않는다(buy_signal_logic.py). 대표 리뷰는 번역 API로 한국어 번역을 함께 보여 준다.

추천 챗봇은 요약 결과를 활용하는 보조 기능이다. 게임별 태그·한줄평·장단점·추천 대상을 카탈로그로 전달하고, 목록에 없는 게임이나 게임 추천 외 질문은 거절하도록 지시했다. 요청은 IP당 분당 10회로 제한되고 카탈로그는 5분간 캐시된다(chat.py; README §3-7). 프롬프트 지시만으로 모든 범위 이탈이 막힌다는 검증은 하지 않았다.

6. 주요 기능과 실제 시연 화면

이 장의 그림 3~10은 2026-06-07 저장소에 커밋된 실제 서비스 화면 캡처(목록 1장, Dark Souls III 상세 1장)를 기능별로 세로로 잘라 실은 것이다. 확대나 내용 보정은 하지 않았으며, 화면의 점수와 가격 상태는 캡처 당시의 표시다. 발표자료 슬라이드 3에 실린 두 화면은 이 캡처의 윗부분과 같은 이미지다.

6.1 게임 목록과 상세 화면 진입

그림 3의 목록 화면은 상단 배너, 게임 제목 검색과 장르·정렬 선택, 게임 카드 그리드로 구성된다. 각 카드에는 표지, 제목, 별점, "AI 리뷰 요약 보기" 버튼이 있고, 오른쪽 아래 원형 버튼으로 챗봇을 연다. 화면의 "전체 게임 리뷰 100개"는 당시 서비스에 등록된 게임 수다.

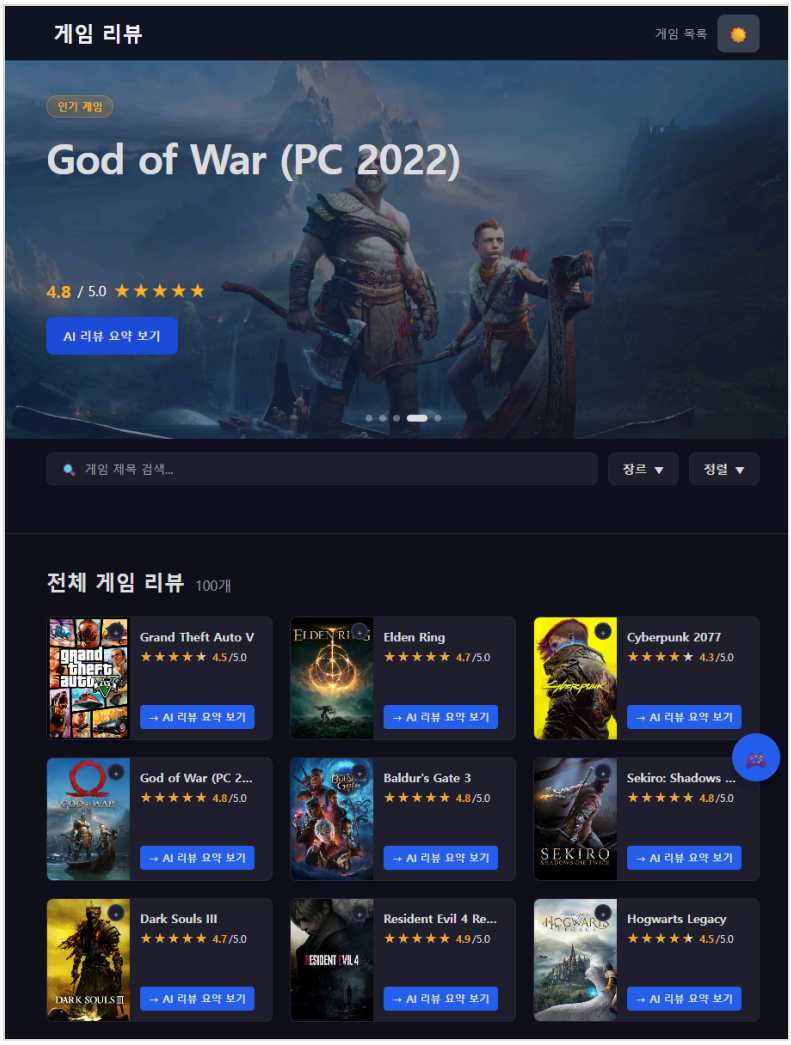


그림 3. 게임 목록 화면 (실제 서비스 캡처)

카드를 누르면 그림 4의 상세 화면으로 이동한다. 상단에 장르 태그, 별점, AI 한줄평이 보이고, "다른 게임과 비교"로 비교 화면에 갈 수 있다. 아래의 "리뷰 토픽"은 리뷰에서 자주 언급된 주제(진행 장벽, 보스, 전투, 그래픽 등)를 보여 준다.

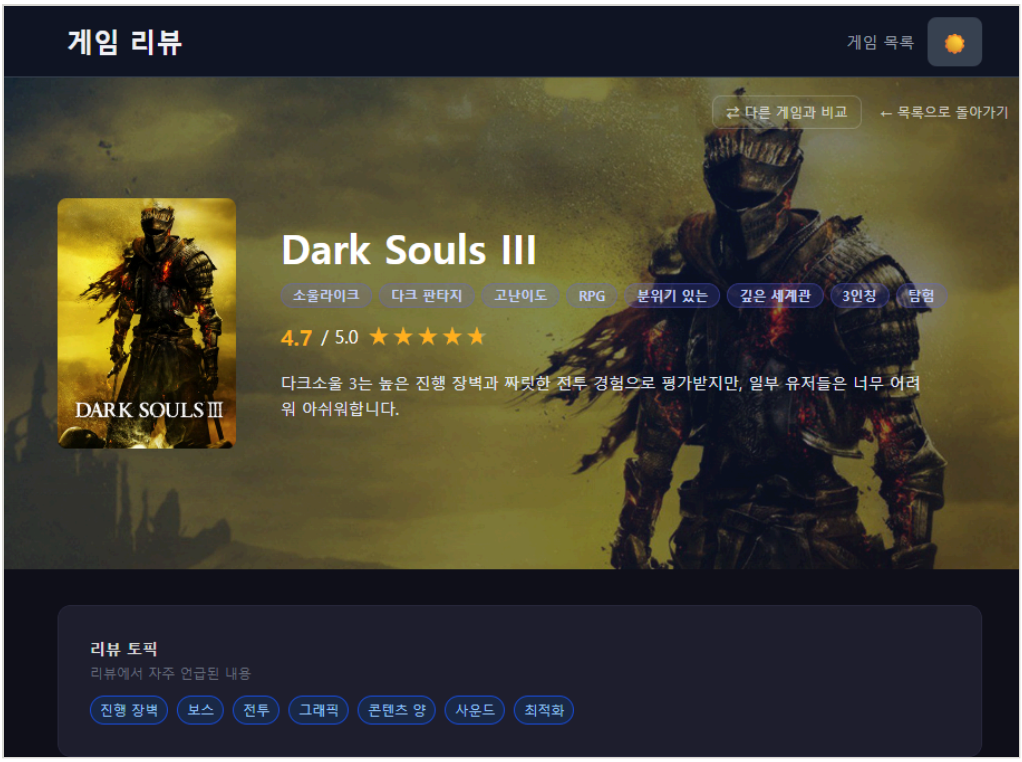


그림 4. 게임 상세 화면 상단: 장르 태그, 별점, 한줄평, 리뷰 토픽 (실제 서비스 캡처)

6.2 유저·평론가 요약과 대표 리뷰

그림 5는 유저 리뷰 요약과 비평가 리뷰 요약을 나란히 보여 준다. 두 요약은 서로 다른 근거로 따로 생성되며, 이 캡처에서는 각각 "매우 긍정적, 94%"와 "매우 긍정적, 89%"로 표시되었다. 유저 요약 본문에는 몹 배치와 맵 디자인에 대한 불만 문장이 두 번 반복된다. 근거에는 맞더라도 문장 중복을 걸러내는 편집 품질이 아직 부족했다는 사례로 10장에서 다시 다룬다.

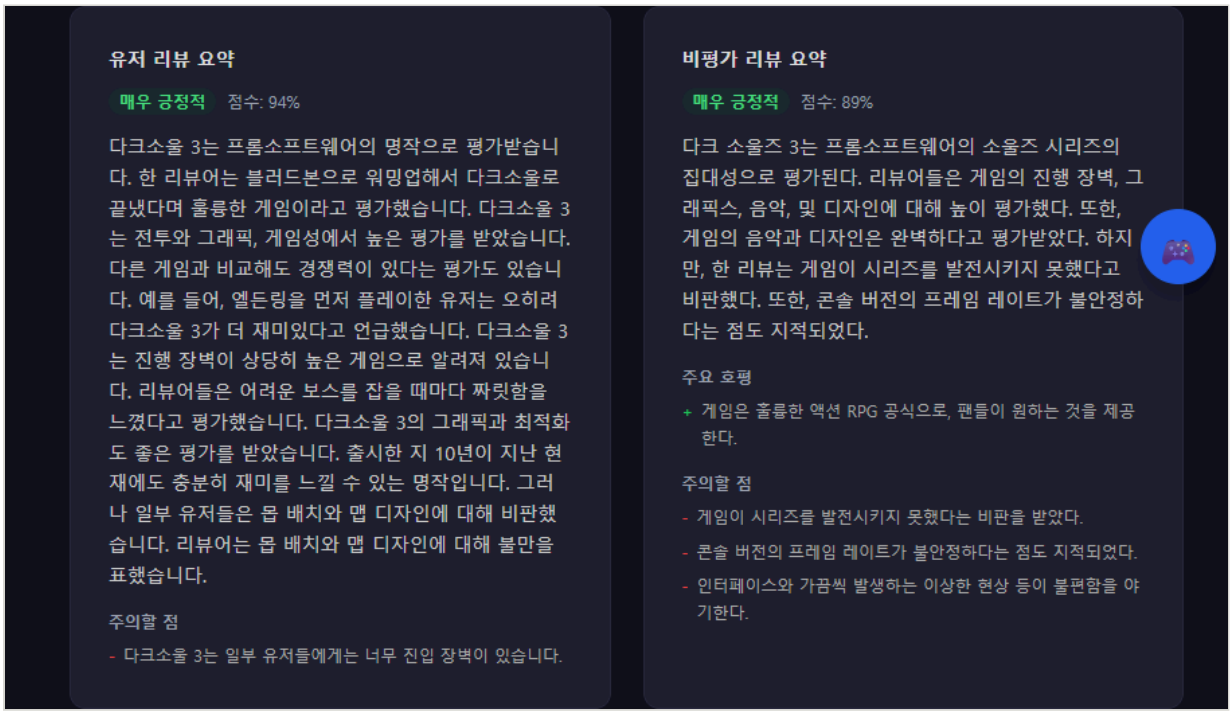


그림 5. 유저·비평가 리뷰 요약 (실제 서비스 캡처)

그림 6은 Steam과 Metacritic 대표 리뷰를 출처별로 보여 준다. 각 카드에는 출처 배지와 리뷰 번호, "원문 보기" 링크가 있고 외국어 리뷰에는 AI 번역 표시가 붙는다. 요약만으로 판단하기 어려운 사용자가 원래 리뷰의 문맥을 확인하는 경로다.



그림 6. 플랫폼별 대표 리뷰와 원문·번역 연결 (실제 서비스 캡처)

6.3 장단점·항목별 분석·플레이타임별 여론

그림 7은 장점·단점 목록과 종합 평가(매우 긍정적, 94%)다. 종합 평가의 등급 이름은 계산된 점수를 Steam식 9단계 구간에 대응시킨 것이며, Steam 공식 등급을 그대로 옮긴 것은 아니다(README §8).



그림 7. 장점·단점과 종합 평가 (실제 서비스 캡처)

그림 8은 5축 레이더(화면에서는 게임플레이 축을 "재미"로 표시한다)와 대표 강점("재미"), 공통 축 밖에서 두드러진 반응(스토리/캐릭터, 음향)을 보여 준다. 5.2에서 설명했듯 레이더는 점수의 크기를, 대표 강점 문구는 언급량 기반 판정을 따른다.



그림 8. 항목별 레이더와 대표 강점·눈에 띄는 반응 (실제 서비스 캡처)

그림 9의 플레이타임별 여론은 이 게임의 구간 경계를 약 30시간 이하, 30~68시간, 68시간 이상으로 나누고, 구간별 긍정 비율(90%·95%·97%)과 요약 문단, 근거 목록을 보여 준다. 이 비율은 이 게임 캡처의 값이며, 다른 게임에서도 플레이할수록 평가가 오른다는 뜻은 아니다.



그림 9. 플레이타임 구간별 여론 (실제 서비스 캡처)

6.4 구매 타이밍과 추천 대상

그림 10의 구매 타이밍 영역은 가격과 할인 정보가 "갱신 대기 중"으로 표시되어 있다. 캡처 당시 가격 스냅샷이 비어 있었다는 뜻이므로, 이 화면은 구매 신호가 실제로 판정된 사례가 아니다. 아래의 "이런 사람에게 추천"은 리뷰에서 반복된 긍정 근거로 정리한 추천 대상(도전적인 게임을 좋아하는 플레이어, 액션 게임 팬)을 보여 준다.

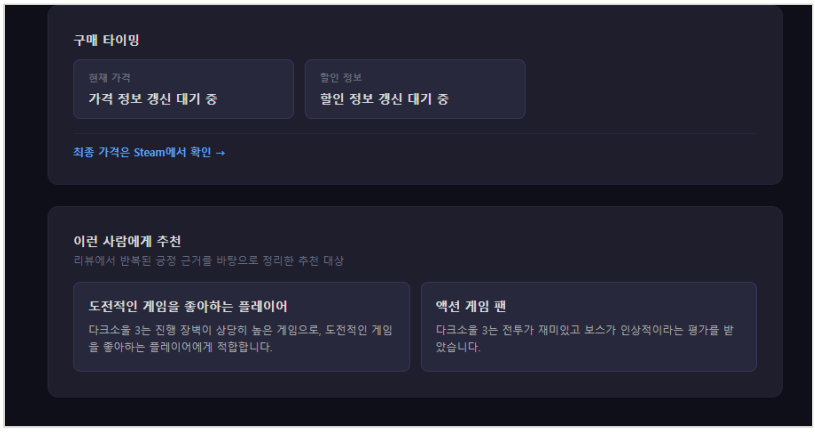


그림 10. 구매 타이밍과 추천 대상 (실제 서비스 캡처)

게임 비교 화면과 추천 챗봇은 구현 문서와 화면의 진입 버튼(그림 3의 챗봇 버튼, 그림 4의 비교 버튼)으로만 확인되며, 해당 화면 캡처는 저장소에 없다. 9장의 달성도 표에서도 이 두 기능은 화면 근거 없이 문서 기준으로 분류했다.

7. 개발 과정과 기술적 문제 해결

7.1 협업과 스프린트

팀원은 4명이며, 개발은 기능 브랜치와 통합 브랜치를 거쳐 main에 병합하는 방식으로 진행했다. 발표자료는 총 커밋 326개를 제시했지만, 같은 슬라이드 차트의 막대 합계는 334개다. 보고서 작성 시점의 저장소에는 전체 커밋 337개가 있고, merge 커밋은 2026-06-10까지 20개다. 발표자료의 두 값은 집계 시점이 기록되지 않아 어느 쪽이 맞는지 정할 수 없다. 그림 11은 발표자료의 주차별 커밋 값을 다시 그린 것이다. 3월 22일~5월 31일 주간 값은 git 이력과 일치하며, 5월 후반에 작업이 몰렸다.

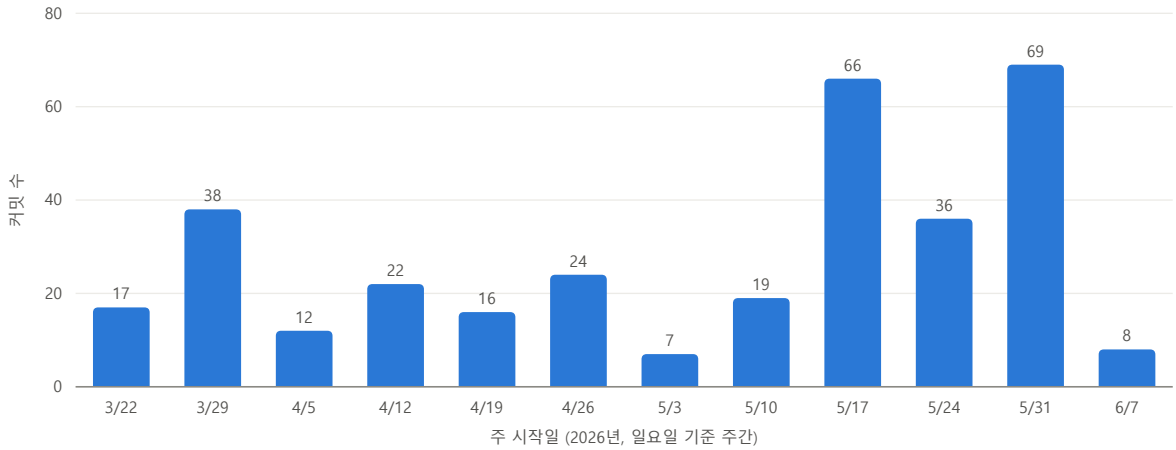


그림 11. 2026년 3~6월 주차별 커밋 수 (발표자료 슬라이드 14 값을 다시 그린 재구성 차트)

스프린트는 5회 진행했다. 발표자료 기준 계획 98pt 중 83pt를 완료했고, 수집·정제·적재(Sprint 1)와 Map-Reduce 요약·목록/상세 화면(Sprint 2)은 100% 완료했다. Sprint 3~5는 번역·검색·품질 평가·플레이타임 분석·구매 신호·챗봇 같은 고도화 작업으로 달성률이 81%, 78%, 60%였다(발표자료 슬라이드 15). DB 스키마 변경도 스프린트 단위 마이그레이션 파일로 누적되었다.

7.2 주요 기술 문제와 해결

개발 노력의 대부분은 요약 품질 문제를 고치는 데 들어갔다. 발표자료는 이를 STAR 방식의 6개 챌린지로 정리했다(PRESENTATION §2).

- **근거 무결성과 환각 차단(A1)**: 리뷰를 그대로 요약시키면 다른 게임 이야기를 지어내거나, Map이 깨진 JSON을 내서 파이프라인이 멈췄다. 근거는 청크 안의 실제 리뷰에서만 뽑게 하고, Reduce 프롬프트에 대상 게임 정보를 고정했으며, 4.3의 JSON 복구 계층을 두었다. LLM을 규칙 기반 추출기로 바꾸는 대신, LLM을 근거 생성의 중심에 두고 검증·복구를 보조 장치로 붙이는 방향을 택했다.
- **톤·표현 정확성(A2)**: 근거가 맞아도 "어렵지만 재밌다"가 단점으로 분류되거나, 근거 없는 미사여구, 어색한 조사, 스포일러가 노출되었다. 긍부정 기준을 만족도로 바꾸고 일반론 제거, 표현 정제, 스포일러 가림 규칙을 추가했다.

- **Reduce 신뢰성(B)**: 5.1의 점수 기준값 고정, 기능별 4회 호출, 무료 한도 대응으로 해결했다.
- **강·약점 판정 재설계(C)**, **증분 요약 품질 보호(D)**, **플레이타임 표본 편향(E)**: 각각 5.2, 5.3, 4.2에서 설명했다.

고친 결함은 합성 입력으로 재현해 회귀 테스트로 고정했다. 그래서 프롬프트나 모델을 바꿔 과거 오류가 다시 생기면 테스트가 실패한다(8.6).

7.3 발표 이후 변경

발표 이후 저장소에는 두 변경이 있었다. 2026-06-11에 단일 VM 운영용 Compose 설정을 추가했고, 2026-06-12에 Reduce 재실행 경로의 오류를 수정하면서 15번 DB 마이그레이션을 초기화 설정에 추가했다(git 커밋 2ad582a). 따라서 README의 "Compose에는 14번까지만 마운트"라는 설명은 현재 설정과 다르다. 이 변경은 발표 당시 기능에 포함하지 않으며, 기존 운영 DB에 15번이 적용되었는지는 확인하지 않았다.

8. 실험·검증 방법과 결과

8.1 평가 대상과 조건

이 장의 수치는 모두 2026년 6월에 생성·평가된 결과다. 보고서 작성 시에는 저장된 결과 파일에서 다시 계산해 일치를 확인했을 뿐, 실험을 재실행하지는 않았다. 근거 충실도는 Google Gemini 계열 모델이, 유용성은 GPT-5.5가 판정했다. Gemini 판정 모델의 정확한 버전은 자료에 오타로 적혀 있어 확인하지 못했다. 요약을 만든 모델(Groq llama-4-scout)과 다른 모델로 채점해 자기 채점 편향을 줄이려는 선택이지만, 판정 모델 자체의 오류는 남는다. 판정의 반복성은 코어셋 20게임을 정상 근거로 다시 채점했을 때 기존 결과를 19게임에서 그대로 재현한 기록이 있다(실험 README).

대부분의 실험은 코어 평가셋 20게임을 사용했다. 기존 충실도(저·중·고), 장르, 리뷰량을 고려해 고른 게임(저 5, 중 8, 고 7)이며 무작위 표본은 아니다. 통합 요약 충실도는 발표 당시 이미 생성되어 있던 100개 게임의 요약을 대상으로 했다(PRESENTATION §3). 실험 문서는 결과를 1~4층으로 묶었는데, 발표자료와 실험 문서의 층 번호가 서로 다르다(발표자료는 2층=점수 정확성, 3층=안정성). 이 보고서는 층 번호 대신 이름으로 부른다.

8.2 근거 충실도

RAGAS의 근거 충실도(faithfulness)는 요약 문장을 검증 가능한 주장으로 나누고, 각 주장이 함께 제공된 근거로 뒷받침되는 비율을 계산한다(발표자료 슬라이드 10). 근거로는 Reduce가 실제로 받은 Map 근거와 대표 인용을 넣었다(결과는 표 7). 이 값은 "요약이 제공된 근거에 맞는가"를 측정하며, 원문 전체를 빠짐없이 담았는지나 구매에 유용한지는 측정하지 않는다.

표 7. 근거 충실도 결과 (판정: Gemini, 저장 결과 재계산 일치)

평가 대상	표본과 근거 조건	평균	기타
통합 요약(한줄평·장단점)	100게임, Reduce 입력 근거	0.931	중앙값 1.000, 최저 0.625, 0.8 이상 90게임
평론가 요약	코어셋 20게임, 전체 근거	0.942	중앙값 1.000
유저 요약	코어셋 20게임, 전체 근거	0.906	중앙값 0.975
Map 근거(출처 원문 대조)	20게임×청크 4개, 해당 리뷰 원문	0.940	0.9 이상 72/80(90%)
Map 근거(전체 리뷰 대조)	같은 80청크, 게임 전체 리뷰	0.941	유효값 79개, 0.9 이상 64/80

통합 요약은 100게임 평균 0.931로, 대부분의 요약 문장이 저장된 근거로 설명 가능했다. 다만 최저 0.625인 게임도 있어 모든 요약이 완벽하다는 뜻은 아니다. 발표 슬라이드는 평론가·유저 결과를 "0.94"로 크게 표시했지만, 실제 값은 평론가 0.942, 유저 0.906이다.

지표가 정말 근거와의 연결을 재는지 확인하기 위해, 100게임 전체가 아닌 코어셋 20게임에서 요약은 그대로 두고 근거만 다른 게임의 것으로 바꿔 다시 채점했다. 평균은 0.868(코어셋 20게임의 정상 조건, 100게임 평균과 표본이 다름)에서 0.013으로 떨어졌고 20게임 모두 하락했다(실험 문서: 상대 −98.5%, Wilcoxon p=7.8e-5). 근거를 끊으면 점수가 무너진다는 것은 이 지표가 문장의 그럴듯함이 아니라 근거와의 연결에 반응한다는 증거다. 다만 모호한 요약 문장은 무관한 근거로도 일부 지지로 판정되는 사례가 있었다.

Map 근거의 충실도는 요약의 토대를 확인하기 위해 측정했다. 게임마다 주장이 많은 청크 4개를 골라, 그 청크에 연결된 리뷰 원문만으로 대조하는 방식과 게임 전체 리뷰로 대조하는 방식을 함께 썼다. 출처 원문 대조에서 평균 0.940, 80청크 중 72개가 0.9 이상이었다. 낮은 점수 일부는 수동 검토 결과 원문에 해당 내용이 실제로 있었고, 판정 모델이 지지를 놓친 경우로 보고되었다. 이 결과는 검토한 80청크에 한정되며, 이때 평가한 Map 결과를 어느 모델이 만들었는지는 문서 간 기록이 엇갈려 특정하지 않는다.

8.3 점수 안정성

안정성과 정합성 실험은 LLM 판정 없이 점수를 직접 계산했다. 두 실험의 조건과 결과를 표 8에 먼저 정리한다.

표 8. 안정성·정합성 실험의 조건과 결과 (평균값은 저장 결과 재계산 일치, 18/20은 분석 문서 보고값)

실험	표본	방법	결과
점수 안정성	입력 기록이 있는 5게임	Map 고정, Reduce 5회 반복 후 점수 산포 계산	총점 표준편차 평균 0.98/100, 항목 0.12/10, 2게임은 5회 모두 같은 총점
점수 정합성	코어셋 20게임	점수를 0~1로 정규화해 Steam 추천율과의 차이를 1에서 뺌	총점 0.909, 항목 0.842, 판별력 점검 18/20게임 통과

같은 근거에서 점수가 얼마나 흔들리는지 보기 위해, 저장된 Reduce 입력 기록을 사용해 Map은 고정하고 Reduce만 5게임에서 각각 5회 반복했다. 총점(0~100) 표준편차는 평균 0.98, 항목 점수(0~10) 표준편차는 평균 0.12였다. 5게임 중 2개(The Witcher 3, Death Stranding)는 5회 모두 총점이 같았고, 가장 크게 흔들린 게임(Dragon Age: The Veilguard)의 표준편차는 2.53이었다. 이 5게임에서는 평가가 비교적 엇갈리는 게임에서 LLM 보정값이 더 흔들려 변동이 컸다.

따라서 "같은 입력이면 같은 점수"는 정확한 표현이 아니다. "근거를 고정하면 점수는 거의 안정적이고 변동은 작다"가 결과에 맞는 표현이다. 원문부터 Map까지 다시 실행하는 전체 재현성은 측정하지 않았고,

그 경우 변동은 더 클 것이다. 입력 기록이 있던 5게임만의 결과이기도 하다.

8.4 점수와 추천 분포의 정합성

정합도는 "1 − |실제 추천율 − 정규화한 점수|"로, LLM 없이 산술로 계산했다. 코어셋 20게임에서 총점 정합도 평균은 0.909, 항목 정합도 평균은 0.842였고 20게임 모두 5개 항목에 데이터가 있었다. 총점은 처음부터 추천률을 기준값으로 쓰도록 설계했으므로 높게 나오는 것이 자연스럽다. 근거에서 계산하는 항목 점수가 항목별 추천률과 0.842로 맞는다는 점이 더 의미 있는 결과다. 단순 단일 프롬프트 요약은 이런 항목 점수 자체를 만들지 않으므로 이 축에서는 비교 대상이 없다.

지표의 판별력은 코어셋 실제 리뷰 5,855건을 합친 조건에서 네 가지 점검(정상 입력, 반대 입력, 무작위 입력, 단조성)을 모두 통과했다. 게임별로는 20게임 중 18게임이 통과했고, 나머지 2게임은 추천율이 50%에 가까워 반대 입력 점검의 기준에 구조적으로 도달하기 어려운 경우였다. 정합도가 낮았던 The Outer Worlds는 요약 당시 추천율 82.4%가 이후 크롤링으로 약 59%까지 바뀌어 정합도 약 0.77이 되었다. 추천율 변화가 작았던 Battlefield 2042는 약 0.98을 유지했다. 점수 결함보다는 요약과 데이터의 시점 차이로 해석되지만, 다시 요약하면 회복된다는 예상은 Groq 한도 때문에 확인하지 못했다.

8.5 유용성 비교

유용성은 같은 게임에 대해 단순 단일 프롬프트 요약(베이스라인)과 파이프라인 요약을 출처와 순서를 숨기고 비교했다. GPT-5.5 판정자 하나가 "실용적 구매 결정자"와 "구체성 추구자" 두 관점에서 코어셋 20게임을 각각 판정했으므로 판정 수는 40이다(표 9). 실제 사용자 실험은 아니다.

표 9. 유용성 쌍 비교 결과 (판정: GPT-5.5, 40판정)

관점	베이스라인 선택	파이프라인 선택	동률
실용적 구매 결정자	18	2	0
구체성 추구자	11	9	0
합계	29	11	0

구매 결정 관점에서는 장점·단점·구매 위험·적합 대상을 넓게 정리한 베이스라인이 18:2로 우세했다. 구체성 관점에서는 11:9로 비슷했고, 파이프라인이 선택된 이유는 대부분 구체적인 조작 방식, 버그, 가격 같은 인게임 세부 정보였다. 구체성 관점에서 두 요약의 순서를 바꿔 다시 판정하자 20게임 모두 같은 요약이 선택되어, 이 관점의 판정은 제시 순서에 흔들리지 않았다. 순서 교체 점검은 이 관점에만 했으며, 발표자료의 "게임마다 3번 평가"는 원자료로 확인되지 않는다.

파이프라인의 출발점은 요약 API(Groq)에 보내는 토큰을 줄이는 것이었지만, 이 비교에서는 토큰·비용·응답 시간을 측정하지 않았다. 따라서 "비용을 줄이면서 유용성을 지켰다"고 결론 내릴 수는 없다. 이 실험이 분명히 보여 준 것은 근거에 충실한 것과 구매 판단에 필요한 정보를 충분히 주는 것이 다른 문제라는 점이다.

8.6 회귀 테스트

회귀 테스트는 LLM을 호출하지 않고, 잘린 JSON·형식 위반·톤 오류·스포일러 같은 LLM 출력 결함을 합성 입력으로 넣어 복구·정제·점수 코드가 올바르게 동작하는지 확인한다. 발표 당시 102개가 통과했고, 보고서 작성 시 같은 102개를 로컬(Python 3.14.3, pytest 9.0.3)에서 다시 실행해 모두 통과했다(48.02초,

발표자료의 3.46초와는 실행 환경이 다르다). 이 테스트는 생성 품질 전체나 서비스 전체 흐름을 대신하지 않으며, 별도의 CI 자동 검사는 도입하지 않았다.

9. 성과와 목표 달성도

9.1 요구사항별 달성 결과

표 10은 2장의 요구사항을 같은 이름으로 최종 구현 상태와 대조한 것이다. 발표 문서의 정리를 따랐으며, 세부 인수 조건을 이번에 다시 시험한 것은 아니다. 15개 범주 중 완전 달성 5개, 부분 달성 7개, 미달 3개다.

표 10. 요구사항별 달성 결과

요구사항	달성	구현 내용과 남은 범위
리뷰 데이터 수집 파이프라인	완전	Steam·Metacritic 수집에서 DB 적재까지 연결
수집 데이터 정제 및 DB 저장	완전	전처리, 점수 정규화, 중복 없는 저장
AI 기반 리뷰 핵심 요약 생성	완전	근거 기반 한줄평·장단점·키워드
게임 목록 및 상세 페이지	완전	카드 목록과 게임별 상세 화면
다국어 리뷰 자동 번역	완전	번역 API와 원문·번역 전환
게임명 통합 검색 및 자동 완성	부분	제목 검색 구현, 자동완성 미구현
상세 조건 필터링 탐색	부분	장르·정렬·빈 결과 안내 구현, 점수 조건 필터 미구현
리뷰 출처 및 신뢰도 배지	부분	플레이타임을 구간 분석으로 활용, 개별 리뷰 배지 미구현
텍스트 위주(무평점) 리뷰 자동 요약	부분	무평점 평론을 요약에 반영, 개별 리뷰 요약 화면 미구현
사용자 취향 기반 개인화 추천	부분	입력 기반 추천 챗봇, 취향 저장 미구현
인기/신작 게임 주기적 자동 동기화	부분	기존 게임의 일일 갱신, 신작 발굴·등록 미구현
리뷰 필터 및 정렬	부분	게임 목록 정렬, 리뷰 단위 출처 필터 미구현
플랫폼 내 자체 유저 리뷰 작성	미달	회원·로그인 미도입
실시간 게임 인기 차트	미달	미착수
검색 기반 게임 정보 실시간 등록	미달	DB에 없는 게임의 즉시 등록 미구현

수집 → 정제·적재 → AI 요약 → 목록·상세·번역으로 이어지는 핵심 경로는 모두 완성했다. 발표 문서는 미달 3건이 모두 낮은 우선순위였다고 설명했지만, 백로그에서 검색 기반 실시간 등록은 High, 실시간 인기 차트는 Medium이었다. 부분 달성인 인기·신작 동기화도 High였다. 핵심 요약 기능은 완성했지만, 카탈로그를 스스로 넓히는 요구는 충족하지 못했다.

9.2 기술적 성과

첫째, 리뷰 ID를 가진 근거를 중심으로 수집·요약·점수·화면을 연결했다. 화면의 한줄평·장단점·추천 대상·점수가 인용 근거를 갖도록 설계했고(README §2), 그 충실도는 8.2의 표본 평가로 확인했다. 저장된 Reduce 입력 기록으로 사후 평가와 재실행도 할 수 있다.

둘째, 품질을 하나의 수치로 합치지 않고 근거 충실도·안정성·정합성·유용성으로 나누어 측정했다. 그 결과 근거 충실도와 정합도는 높지만 구매 결정 관점의 유용성은 단순 요약보다 낮다는 약점이 드러났다. 측정을 나눴기 때문에 다음 개선 방향이 분명해졌다.

셋째, 구조화된 요약을 플레이타임 분석, 게임 비교, 구매 신호, 추천 챗봇 등 여러 기능에서 재사용했다. 이 중 비교와 챗봇은 문서로만 확인되며, 구매 판단 시간이나 만족도 향상은 측정하지 않았다.

10. 한계와 향후 개선

10.1 데이터와 출력 품질

요약은 게임당 기본 200건의 선별 표본을 사용하므로 전체 리뷰를 담지 못한다. 도움 수·플레이타임 우선 선택과 길이 필터 때문에 짧은 핵심 리뷰나 갓 올라온 리뷰는 표본에서 밀리기 쉽다. 로컬 Map 모델이 작아 한국어·영어 외 언어는 처리 품질이 낮아, 다른 언어권 여론은 반영하지 못한다(PRESENTATION §6). 서비스 카탈로그도 미리 정한 100개 게임에 머물렀다. 출력 면에서는 그림 5의 반복 문장처럼 근거에는 맞지만 읽기 품질이 떨어지는 문장이 남아 있고, 8.5에서 드러난 구매 판단 정보의 부족도 해결해야 한다. 표본 상한 확대나 누락분 재처리, 중복 문장 제거, 구매 위험·적합 대상 항목의 보강이 다음 과제다.

10.2 평가의 한계

충실도와 유용성은 각각 단일 판정 모델의 결과이며 사람 검토는 일부 저점 사례에만 있었다. 안정성은 Map을 고정한 5게임 결과이고, 정합성은 요약과 데이터의 시점 차이에 민감하다. 토큰·비용·응답 시간과 실제 사용자 가치는 측정하지 않았다. 또한 RAGAS 평가는 요약이 생성된 뒤 따로 돌리는 사후 평가여서, 품질이 낮은 요약이 그대로 노출될 수 있다. 향후에는 생성 직후 충실도를 검사해 기준 미달이면 다시 생성하거나 신뢰도를 표시하고, 여러 판정 모델과 사람 평가, 비용 기록을 함께 수집해야 한다.

10.3 기능과 운영

우선 보완할 기능은 검색 기반 실시간 등록과 인기·신작 자동 편입이다. 지금은 DB에 있는 게임만 볼 수 있어 카탈로그 밖의 게임을 찾으면 결과가 없다. 로그인과 자체 리뷰, 취향 누적 기반 추천도 남아 있다. 운영 면에서는 가격이 Redis에만 있어 초기화 시 비는 문제, 평론가 리뷰의 수동 수집, Groq 한도 소진, 모든 출처를 허용하는 CORS 설정, 기존 DB의 마이그레이션 적용 여부를 정리해야 한다(README §8). 현재 공개 서비스와 외부 시연 영상 링크도 확인되지 않으므로, 재배포 후 수집부터 조회까지의 전체 흐름을 실제 환경에서 다시 확인해야 한다.

11. 결론

게임 리뷰 AI 요약 서비스는 리뷰 ID를 가진 근거를 중심으로 수집, 근거 추출, 관점별 요약, 점수 산출, 화면 제공을 하나의 흐름으로 구현했다. 발표 당시 저장된 평가에서 통합 요약의 근거 충실도는 100게임 평균 0.931이었고, 근거를 다른 게임 것으로 바꾸면 0.013으로 떨어져 지표가 근거 연결에 반응함을 보였다. Map을 고정한 Reduce 반복의 총점 표준편차는 평균 0.98/100이었고, 점수와 추천 분포의 정합도는 총점 0.909, 항목 0.842였다.

반면 구매 결정 관점의 유용성은 단순 요약에 18:2로 뒤졌다. 근거에 충실하게 만드는 것만으로는 구매 판단에 필요한 정보의 폭이 확보되지 않는다는 것이 이 프로젝트의 가장 중요한 교훈이다. 요구사항 15개 범주 중 핵심 경로를 포함한 5개를 완전히, 7개를 부분적으로 달성했다. 다음 단계는 근거의 구체성을 유지하면서 정보의 폭을 넓히는 출력 보강, 생성 직후 품질 검사, 카탈로그 자동 확장, 비용과 실제 사용자 가치의 측정이다.

12. 참고자료

프로젝트 저장소(발표 당시 공개 주소: github.com/euden112/make-review-web)의 다음 문서와 코드를 근거로 했다. 주장별 파일·위치 대응표, 그림 출처 목록, 보고서 작성 시 재확인 기록은 이 보고서와 함께 제출하는 폴더(docs/final-report/sources/)에 있다.

- README.md — 서비스 소개, 문제 정의, 핵심 차별점, 기술 스택, 운영 방식, 알려진 주의 사항
- docs/ARCHITECTURE.md — 데이터 흐름, 실행 경로, 점수 산출, 저장·캐시 구조, 배포, 알려진 한계
- docs/MAP_REDUCE.md — 청킹, 근거 형식, JSON 복구, Reduce 4회 호출, 점수 수식
- docs/steam_crawling.md, docs/playtime_improvement.md — 수집·정제 방식, 플레이타임 표본 선택
- docs/PRESENTATION.md — 최종 발표 정리(기술 챌린지, 품질 증명, 개발 과정, 달성도, 향후 과제)
- 최종 발표자료(게임리뷰AI요약_최종발표, 18장) — 슬라이드 1~18
- ai-pipeline/experiments/ — 평가셋 구성과 실험별 분석 문서(충실도, 안정성, 정합성, 유용성) 및 결과 CSV
- 소스 코드: ai-pipeline/ai_module/map_reduce/(sampler.py, pipeline.py, map_local.py, reduce_api.py), backend/app/(api/v1/chat.py, services/buy_signal_logic.py), docker-compose.yml